Master EID²

Université Sorbonne Paris Nord

UE Science des Données

Fiche TP

Le but de ce TP est de créer une fonction (ou une classe) *analyse(X, Y)* capable d'analyser un jeu de données : décrire ses variables, détecter des clusters de données homogènes, puis prédire les labels des observations. Cette fonction prend en entrée une matrice de données X et une liste de labels Y. Les labels ne servent pas à former les clusters : ils servent à évaluer les clusters obtenus, puis à entraîner les classifieurs.

Développez et testez chaque partie sur les données iris (*datasets.load_iris()*).

Toutes les figures ou sous-figures doivent avoir un titre et une légende sur les axes !

A) Description et visualisation des données :

1. La fonction *analyse* doit afficher le nombre de données, de variables et de classes, ainsi que la moyenne, l'écart type, les valeurs min et max de chaque variable.
2. Elle doit aussi afficher une visualisation des données à l'aide d'une ACP (fonction *PCA* de *sklearn*) et une autre avec une ADL (fonction *LinearDiscriminantAnalysis* de *sklearn*). Dans les deux cas, la couleur de chaque point doit dépendre de son label Y. Expliquez brièvement les principes de l'ACP et de l'ADL ainsi que leurs différences.

B) K-means :

1. Écrivez en Python l'algorithme K-means sous la forme d'une fonction. Elle prend en entrée deux paramètres : la matrice des données et le nombre de clusters souhaité. Elle renvoie le numéro de cluster de chaque donnée. Comparez avec la fonction *KMeans* de *sklearn*.
2. Les centres des clusters étant choisis au hasard lors de l'initialisation, le résultat obtenu peut varier d'une exécution à l'autre. Vérifiez que c'est le cas.
3. Utilisez l'indice de Silhouette (*silhouette_score* de *sklearn*) pour stabiliser les résultats et sélectionner automatiquement le nombre de groupes. Pour ce faire, appliquez votre K-means pour un nombre de clusters allant de 2 à 10, 10 fois pour chaque nombre de clusters (soit 90 fois en tout), et renvoyez la solution ayant le meilleur score de Silhouette. Le nombre de clusters retenu correspond-il au nombre de classes dans Y ?
4. Affichez les clusters obtenus sur l'ACP de la partie A, à côté de la figure colorée par les labels Y. Les groupes obtenus sont-ils cohérents avec les classes ?

C) Test de différents algorithmes de clustering :

1. La fonction *analyse* doit proposer un clustering des données. Appliquez votre fonction de la partie B et les fonctions prédéfinies de *sklearn.cluster* pour les algorithmes suivants : « mean shift », « affinity propagation », « spectral clustering » et « DBSCAN ». Lorsque nécessaire, récupérez le nombre de clusters à partir du nombre de classes dans Y.
2. Pour chaque méthode, calculez et affichez les scores internes de Davies-Bouldin et Silhouette, ainsi que les scores externes Adjusted Mutual Info et Adjusted Rand score (voir *sklearn.metrics*). Quelle est la différence entre un score interne et un score externe ?
3. Estimez à partir des résultats précédents la meilleure méthode de clustering. Justifiez bien vos réponses.

D) Test de différents algorithmes de classification :

1. En utilisant la méthode du leave-one-out, la fonction *analyse* doit prédire le label de chaque observation (ligne de X). Appliquez le Plus Proche Voisin et le Classifieur Bayésien Naïf (les vôtres si vous les avez déjà écrits, sinon *KNeighborsClassifier* avec *n_neighbors=1* et *GaussianNB*), puis le Perceptron Multi-Couche (*MLPClassifier*) et les machines à vecteurs de support (*svm.SVC*). Pour les fonctions prédéfinies, utilisez les paramètres par défaut.
2. Pour chaque méthode, calculez et affichez le pourcentage d'erreurs, puis les valeurs de precision, recall et f1-score pour chaque classe (fonction *classification_report*).
3. Estimez à partir des résultats précédents la meilleure méthode de classification. Justifiez bien vos réponses.

E) Utilisation de la fonction :

1. Testez la fonction sur les données iris :

   data = datasets.load_iris()

   Il s'agit de données de taille (longueur et largeur) de pétales et sépales de fleurs (4 variables), regroupées en trois espèces d'iris de 50 individus chacune.

   Commentez les résultats.

2. Importez les données wine :

   data = datasets.load_wine()

   Il s'agit de vins provenant de trois lieux différents. Les 13 variables représentent les quantités mesurées de 13 constituants chimiques présents dans les trois types de vins. 178 vins ont été mesurés, 59 du type 1, 71 du type 2 et 48 du type 3.

   Analysez ces données avec la fonction *analyse*.

3. Les 13 variables de wine n'ont pas la même échelle. Relancez l'analyse après avoir centré et réduit les données (*StandardScaler*). Que constatez-vous pour le clustering et pour la classification ?